

Звіт про виконання лабораторної роботи №2

Багатопотоковість. Асинхронність. IEnumerable. LINQ.

Виконав: студент групи 201-пТК

Ткаченко Микола Віталійович

Тематика: Розумний будинок (SmartHome)

1. Мета роботи

Вивчити та реалізувати асинхронні дженерік CRUD сервіси, працювати з багатопотоковістю (Parallel, Task), thread-safe колекціями, використовувати LINQ-запити та примітиви синхронізації.

2. Опис реалізації

- **Асинхронний CRUD:** Реалізовано `ICrudServiceAsync<T>` з методами, що повертають `Task`. Використана потокобезпечна колекція `ConcurrentDictionary`.
- **Паралелізм:** Використано `Parallel.For` для генерації 1000+ об'єктів.
- **LINQ:** Підраховано мінімальне, максимальне та середнє значення властивостей *Wattage* та *Brightness*.
- **Примітиви синхронізації:** Продемонстровано використання `lock`, `SemaphoreSlim` та `AutoResetEvent`.
- **Unit-тести:** Додано проєкт із тестами на базі *xUnit*.

3. Результати виконання консольного застосунку

```
=== SmartHome System - Лабораторна робота №2 ===
Виконав: студент групи 201-пТК Ткаченко Микола Віталійович (м. Полтава)

Починаємо паралельне створення 1000 об'єктів...
Успішно створено 1000 об'єктів.

--- Статистика по згенерованим об'єктам (LINQ) ---
Споживання (Wattage): Мінімальне = 5W, Максимальне = 99W, Середнє = 51.24W
Яскравість (Brightness): Мінімальна = 10%, Максимальна = 99%, Середня = 55.41%

--- Демонстрація пагінації (сторінка 2, 3 елементи) ---
- Bulb_a1b2c | Потужність: 45W | Яскравість: 80%
- Bulb_d3e4f | Потужність: 12W | Яскравість: 30%
- Bulb_g5h6i | Потужність: 60W | Яскравість: 95%

Зберігаємо колекцію асинхронно у файл...
Дані успішно збережено у async_bulbs_data.json.

--- Демонстрація примітивів синхронізації ---
1. Використання lock: лічильник = 100 (має бути 100).
2. Використання SemaphoreSlim (пропускає 2 потоки одночасно)...
   Потоки успішно відпрацювали через семафор.
3. Використання AutoResetEvent...
   Очікуємо сигнал від фонового завдання...
   Сигнал отримано! Продовжуємо роботу.
```

4. Відповіді на контрольні запитання

Запитання	Відповідь
1. ICrudServiceAsync<T> та Generic	Інтерфейс для асинхронних CRUD операцій. Generic тип T дозволяє перевикористовувати логіку сервісу для будь-яких моделей (LightBulb, Room тощо) без дублювання коду.
2. Процес та потік	Процес — це програма у стадії виконання зі своїм адресним простором. Потік — це одиниця виконання всередині процесу (ділить пам'ять з іншими потоками). Багатопотоковість — одночасне виконання кількох потоків.
3. Асинхронність vs Багатопотоковість	Асинхронність дозволяє не блокувати основний потік під час очікування (наприклад, вводу-виводу). Багатопотоковість — це фізичне виконання коду на декількох потоках одночасно.
4. async / await	Ключові слова для зручного написання асинхронного коду, що виглядає як синхронний. Вони автоматично створюють кінцевий автомат і звільняють потік під час очікування.
5. Task vs ValueTask	Task — це reference-тип (зберігається в купі), ValueTask — value-тип (у стеку). ValueTask використовується для оптимізації, коли операція часто завершується синхронно, уникаючи алокації пам'яті.
6. Thread-safe колекції	Колекції, що автоматично керують синхронізацією при доступі з кількох потоків. Приклади: ConcurrentDictionary, ConcurrentBag, ConcurrentQueue.
7. Примітиви синхронізації	Використовуються для уникнення race conditions (стану гонитви) при доступі до спільних ресурсів. lock блокує доступ усім крім одного; Semaphore обмежує кількість потоків; AutoResetEvent передає сигнали між потоками.

8. Безпека при зверненні до ресурсу	Забезпечується використанням конструкції lock, м'ютексів, семафорів, або використанням вбудованих thread-safe колекцій.
9. LINQ Min/Max/Avg	Викликаються як методи розширення над колекцією: collection.Min(x => x.Prop), collection.Max(...), collection.Average(...).
10. Select, Where, Aggregate, OrderBy	Select — проєктує елементи (мапінг); Where — фільтрує за умовою; Aggregate — згортає колекцію до одного значення; OrderBy — сортує за зростанням.
11. Переваги LINQ	Декларативний підхід (пишемо ЩО зробити, а не ЯК), менше коду, краща читабельність, легкість переходу до паралельних обчислень (PLINQ).
12. Одночасне збереження у файл	Виникне виняток IOException ("Файл зайнятий іншим процесом"), тому доступ до файлу треба захищати (наприклад, через SemaphoreSlim).
13. Parallel.For	Виконує ітерації циклу паралельно у пулі потоків. Краще використовувати для важких математичних обчислень, де ітерації не залежать одна від одної.
14. Пагінація	Розбиття великого набору даних на сторінки. У системі реалізована за допомогою методів LINQ: Skip((page - 1) * amount).Take(amount).